

Enhanced for Loop (also known as For-Each Loop)

The **enhanced for loop** in Java is a simplified version of the traditional `for` loop, designed to iterate over arrays or collections (such as lists or sets) without needing to explicitly manage an index variable.

It is commonly used when you want to access each element in an array or collection directly, making the code cleaner and more readable.

Syntax of Enhanced `for` Loop:

```
for (datatype variable : collection) {  
    // Code to be executed  
}
```

- **datatype** : The type of the elements in the array or collection.
- **variable** : The variable that will hold the current element during each iteration.
- **collection** : The array or collection (such as an `Array`, `List`, `Set`, etc.) you want to iterate over.

The enhanced `for` loop automatically iterates over each element in the array or collection, and you don't need to manually access elements by index.

How It Works:

1. The enhanced `for` loop automatically iterates through the collection or array.
 2. For each iteration, it assigns the current element to the specified variable.
 3. The loop terminates once all elements in the collection have been processed.
-

Example 1: Iterating over an Array

```
public class Main {  
    public static void main(String[] args) {  
        int[] numbers = {1, 2, 3, 4, 5};  
  
        for (int num : numbers) {  
            System.out.println("Number: " + num);  
        }  
    }  
}
```

```
    }  
  }  
}
```

Explanation:

- The loop iterates over the `numbers` array.
- On each iteration, the element of the array is assigned to the variable `num`, which is then printed.
- The loop stops after all elements in the array have been processed.

Output:

```
Number: 1  
Number: 2  
Number: 3  
Number: 4  
Number: 5
```

Example 2: Iterating over a `List` (Collection)

```
import java.util.ArrayList;  
  
public class Main {  
    public static void main(String[] args) {  
        ArrayList<String> names = new ArrayList<>();  
        names.add("Alice");  
        names.add("Bob");  
        names.add("Charlie");  
  
        for (String name : names) {  
            System.out.println("Name: " + name);  
        }  
    }  
}
```

Explanation:

- Here, the enhanced `for` loop is used to iterate over a `List` of strings.
- The variable `name` holds the current element of the list during each iteration, and it is printed.
- The loop terminates after all elements in the list have been processed.

Output:

```
Name: Alice
Name: Bob
Name: Charlie
```

Key Points:

1. **Simplified Syntax:** The enhanced `for` loop eliminates the need for indexing, making the code cleaner and easier to read.
 2. **Automatic Iteration:** The loop automatically iterates through every element in the array or collection without needing explicit index management.
 3. **Read-Only Access:** The enhanced `for` loop provides read-only access to elements. If you want to modify the elements during iteration, use a traditional `for` loop.
 4. **Works with Arrays and Collections:** The enhanced `for` loop works seamlessly with arrays, lists, sets, and other collections that implement the `Iterable` interface.
-

Example 3: Modifying Elements (Traditional `for` loop is preferred)

If you need to modify elements during iteration, you should use a traditional `for` loop, as the enhanced `for` loop does not allow modification of elements directly.

For example, to increment each element in an array:

```
public class Main {
    public static void main(String[] args) {
        int[] numbers = {1, 2, 3, 4, 5};

        for (int i = 0; i < numbers.length; i++) {
            numbers[i] = numbers[i] + 1; // Modify the array elements
        }

        // Print the modified array
        for (int num : numbers) {
            System.out.println("Modified Number: " + num);
        }
    }
}
```

Output:

```
Modified Number: 2  
Modified Number: 3  
Modified Number: 4  
Modified Number: 5  
Modified Number: 6  
Modified Number: 7
```
